

ROS

ROS (ang. Robot Operating System) jak nazwa wskazuje nie jest jednak systemem operacyjnym. Zapewnia wsparcie dla niskopoziomowych urządzeń oraz łatwą komunikację między procesami. Zaletą jest fakt, że przy tworzeniu nowego robota użytkownik najprawdopodobniej nie będzie musiał martwić się o obsługę danych czujników na niskim poziomie. Istnieje duże prawdopodobieństwo, że już ktoś inny napisał taką obsługę i ją udostępnił. Takie podejście pozwala na szybsze przejście do tworzenia bardziej zaawansowanych algorytmów bez konieczności odtwarzania "koła na nowo".

Jest w pełni darmowy zarówno w celach prywatnych, naukowych jak i komercyjny co zwiększa jego popularność.

ROS - System plików

W systemie plików można wyróżnić następujące typy:

- **packages** - podstawowa jednostka organizacji plików, zawiera nody, struktury wiadomości i serwisów, pliki konfiguracyjne z parametrami. Pliki w niej zawarte odpowiadają za realizację ściśle wyizolowanego zadania np. komunikacja ze skanerem laserowym i odczyt danych, wyznaczenie lokalizacji robota, odczyt danych z joysticka, odebranie informacji o joysticku iysterowanie prędkości postępowych i obrotowych robota
- **metapackages** - pozwalają na logiczne zgrupowanie różnych paczek w jedną np. paczka "navigation", "turtlebot3". W porównaniu do **packages** nie zawiera ona nodów, typów wiadomości, serwisów, tylko informację o podstawowej konfiguracji i zależnościach od innych paczek.
- **package manifests** - zawiera podstawowe informacje o paczce takie jak nazwa, wersja, autor, licencja, zależności od innych paczek oraz wiele więcej
- **repositories** - paczki przechowywane są na repozytoriach z kontrolą wersji, a także z wykorzystaniem **bloom**, które umożliwia m.in. instalowanie paczek ROS przy użyciu *apt-get install*.
- **message (msg)** - zawiera informację o strukturze wiadomości
- **service (srv)** - zawiera informację o strukturze serwisu (struktura wiadomości wysyłana przez klienta i odsyłana odpowiedź przez serwer)

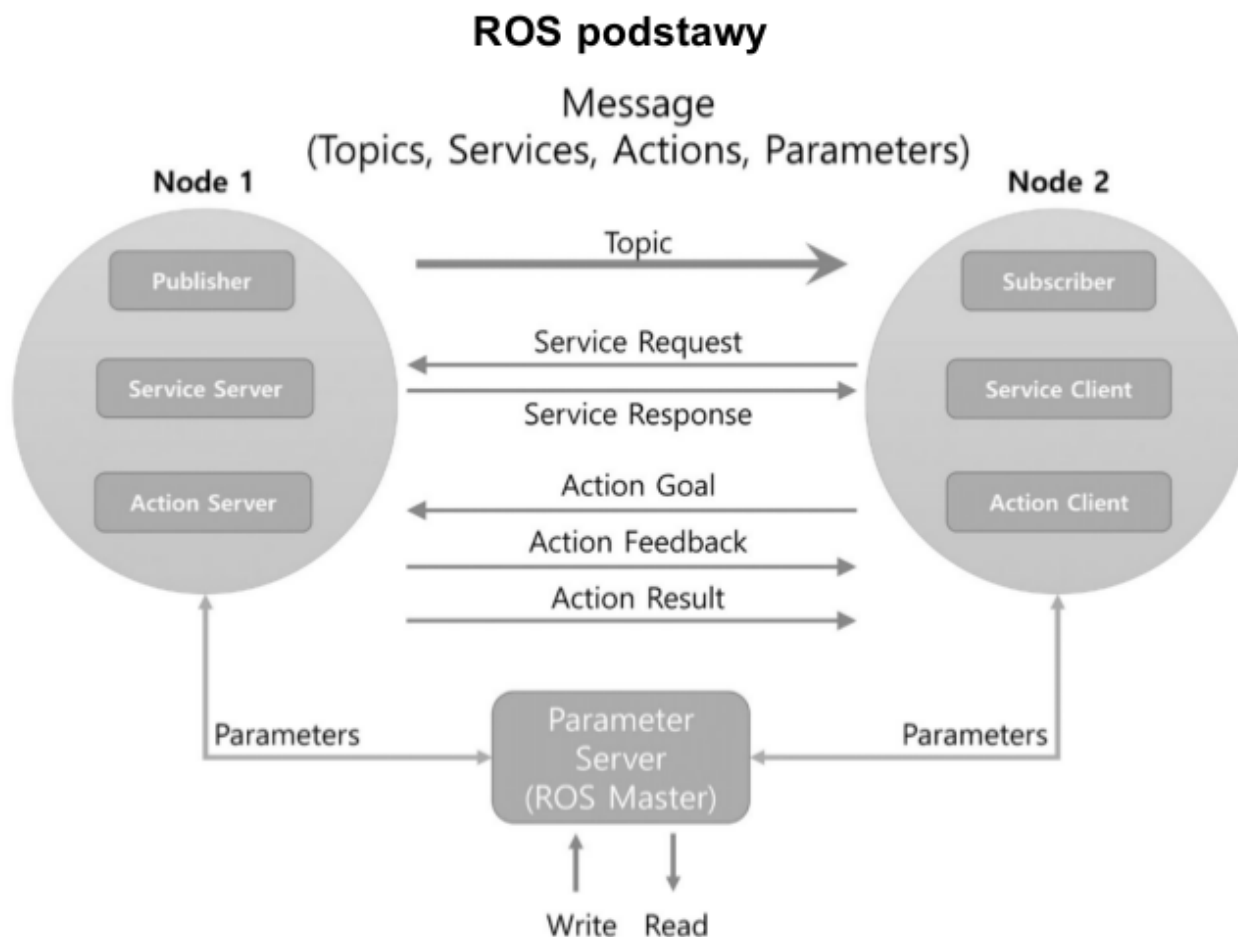
Struktura workspace'a

- **/build**
- **/devel** - zawiera ostateczne pliki wykonywalne, biblioteki, zdefiniowane msg, srv
- **/src** - zawiera paczki użytkownika
 - **/user_package**
 - **/include/user_package** - zawiera pliki nagłówkowe C++
 - **/launch** - zawiera pliki .launch do uruchomienia
 - **/msg** - nowe struktury wiadomości dedykowane dla tej paczki
 - **/scripts** - wykonywalne skrypty
 - **/src** - pliki wykonywalne C++
 - **/srv** - pliki do wymiany informacji dla komunikacji Client Service
 - **CMakeLists.txt** - plik konfiguracyjny budowania paczki
 - **package.xml** - plik konfiguracyjny paczki

ROS Computation Graph

ROS bazuje na architekturze komunikacji peer-to-peer, tzn. każdy węzeł podłączony do sieci może jednocześnie nadawać i odbierać dane od innych węzłów. W skład *Computation Graph* wchodzi:

- **master** - umożliwia komunikację i wymianę informacji. Bez niego nie jest możliwe korzystanie z możliwości ROS.
- **nodes** - proces odpowiedzialny za obliczenia. W systemie robotycznym można wyróżnić np. nody odpowiedzialne za wyznaczenie lokalizacji, nawigację, obsługę kamer, obsługę pojedynczych czujników.
- **parameter server** - baza danych z parametrami. Przykłady: parametry stałe czujników, które są wykorzystywane w node'ach, dokładność dojazdu robota do punktu, opis robota.
- **messages** - są to informacje przekazywane/odbierane przez różne node'y w systemie. Składają się z podstawowych typów (np. int, float, string) jak i bardziej złożonych (np. struktura punktu ze współrzędnymi x,y). Jako przykład można podać przekazanie informacji o położeniu od node'a od lokalizacji do node'a odpowiedzialnego za nawigację.
- **topics** - jeden z nodów nadaje wiadomość na topic'u o określonej nazwie. Każdy node który chce odbierać i wykorzystywać informacje pojawiające się na nim ma taką możliwość. Przykładem może być odbieranie danych z kamery na robocie i przekazywanie ich dalej. Operator robota może odbierać dane z kamery i wyświetlać je u siebie. Jeden z nodów znajdujących się na robocie może być odpowiedzialny za wykrywanie przedmiotów na obrazie.
- **services** - jest to typ komunikacji klient-serwer. Dane przekazywane są do klienta na jego żądanie. W systemie robotycznym może być do zlecenie wykonanie jakiegoś zadania przez robota i konieczne jest oczekiwanie na odpowiedź potwierdzającą wykonanie akcji.
- **bags** - służy do zapisywania i odtwarzania pojawiających się w systemie wiadomości



Master

Roscore jest masterem, który zarządza informacjami o połączeniach dla komunikacji pomiędzy nodami (węzłami) i jest istotnym elementem, który musi być uruchomiony jako pierwszy w celu korzystania z systemu ROS.

ROS master jest uruchamiany poleceniem 'roscore'. Master rejestruje informacje o nodach, takie jak nazwy, topici i serwisy, typy wiadomości, adresy URI i porty. W razie potrzeby informacje te są przekazywane do innych węzłów.

Po uruchomieniu "roscore", uruchamiany jest również "rosout", który jest używany do rejestrowania standardowych logów ROS, takich jak DEBUG, INFO, WARN, ERROR, FATAL, itd. Uruchamiany jest również serwer parametrów, który zarządza parametrami w systemie.

W tym celu w terminalu należy wpisać:

```
roscore
```

Uruchomienie node'a

Uruchomienie przykładowego node'a w terminalu. Jest to symulator robota żółwia, na którym oparte będą dalsze ćwiczenia. Podstawowy sposób uruchomienia node'a w terminalu:

```
roslun nazwa_paczki nazwa_node'a
```

Poniższe node'y uruchomić w terminalu

Uruchomienie node'a następuje przez użycie polecenia:

```
roslun turtlesim turtlesim_node
```

Sterowanie robotem

```
roslun turtlesim turtle_teleop_key
```

Tworzenie node'a - Python

W pojawiających się skryptach do laboratorium do komunikacji z innymi nodami musi pojawić się tylko jedna inicjalizacja node'a (*rospy.init_node*).

```
In [1]: import rospy
```

```
node_name= "test" # UZUPEŁNIĆ własną nazwą  
rospy.init_node(node_name, anonymous=True)
```

```
In [2]: import rosnode
```

```
rosnode.get_node_names()
```

```
Out[2]: ['/turtlesim', '/teleop_turtle', '/rosout', '/test_949_164621634  
2500']
```

Launch file

Pozwala na uruchomienie wielu node'ów na raz wraz z konfiguracją parametrów.

```

<?xml version="1.0"?>
<launch>
  <!-- move_base -->
  <!-- Arguments -->
  <arg name="robot_id" default="default_robot"/>
  <arg name="cmd_vel_topic" default="cmd_vel" />
  <arg name="odom_topic" default="odom" />

  <!-- move_base -->
  <node pkg="move_base" type="move_base" respawn="false" name="move_base" output="screen">
    <param name="base_local_planner" value="teb_local_planner/TebLocalPlannerROS" />
    <rosparam file="$(find tsr)/param/move_base.yaml" command="load" />
    <rosparam file="$(find tsr)/param/local_planner.yaml" command="load" />
    <rosparam file="$(find tsr)/param/global_costmap.yaml" command="load" />
    <param name="global_costmap/robot_base_frame" value="$(arg robot_id)/base_footprint"/>
    <param name="global_costmap/static/map_topic" value="/$(arg robot_id)/map"/>
    <param name="global_costmap/obstacle_layer/laser/sensor_frame" value="$(arg robot_id)/base_scan"/>
    <param name="global_costmap/obstacle_layer/laser/topic" value="/$(arg robot_id)/scan"/>

    <rosparam file="$(find tsr)/param/local_costmap.yaml" command="load" />
    <param name="local_costmap/robot_base_frame" value="$(arg robot_id)/base_footprint"/>
    <param name="local_costmap/obstacle_layer/laser/sensor_frame" value="$(arg robot_id)/base_scan"/>
    <param name="local_costmap/obstacle_layer/laser/topic" value="/$(arg robot_id)/scan"/>

    <remap from="cmd_vel" to="$(arg cmd_vel_topic)"/>
    <remap from="odom" to="$(arg odom_topic)"/>
  </node>
</launch>

```

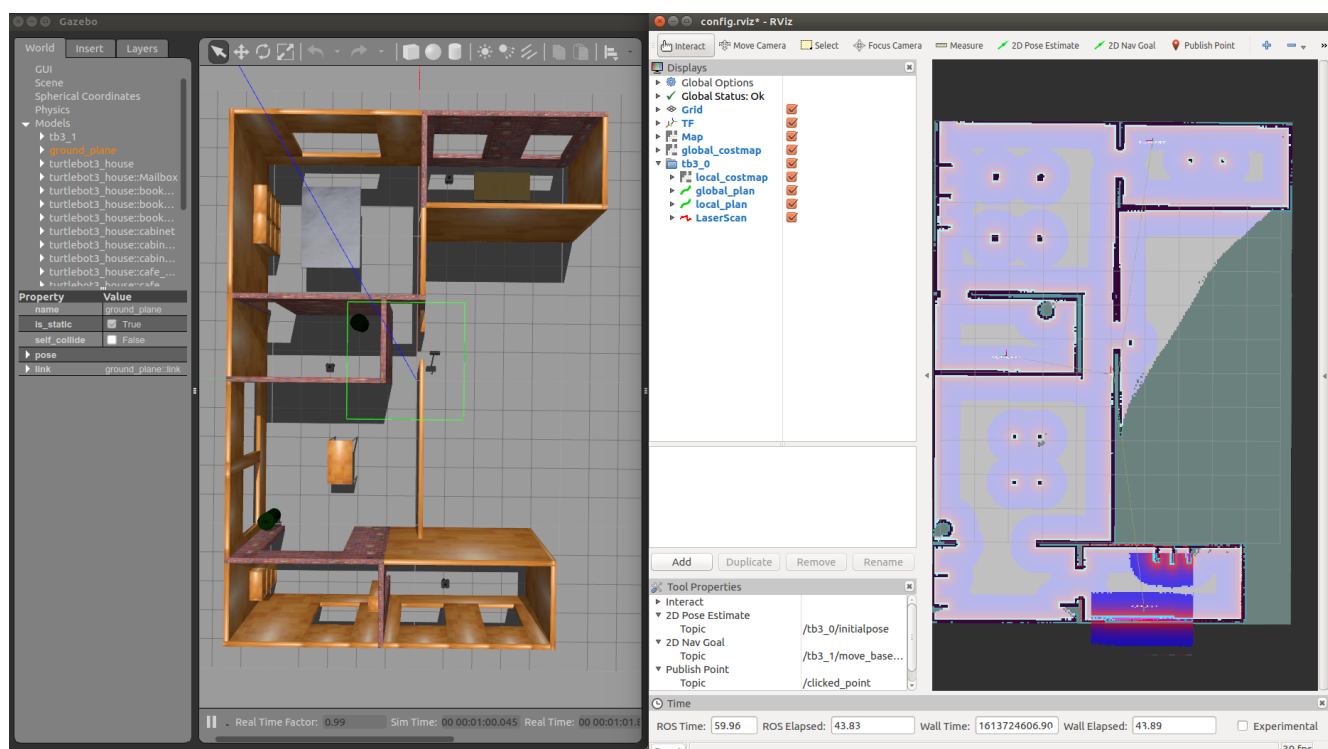
Poniższe polecenia uruchomić w oddzielnych terminalach. Podstawowy sposób uruchomienia node'a w terminalu:

roslaunch nazwa_paczki nazwa_launch'a

Przykładowe launch'e do uruchomienia w nowych terminalach:

roslaunch multirobot_nav multi_turtlebot3.launch roslaunch multirobot_nav multi_nav.launch

GAZEBO, RVIZ

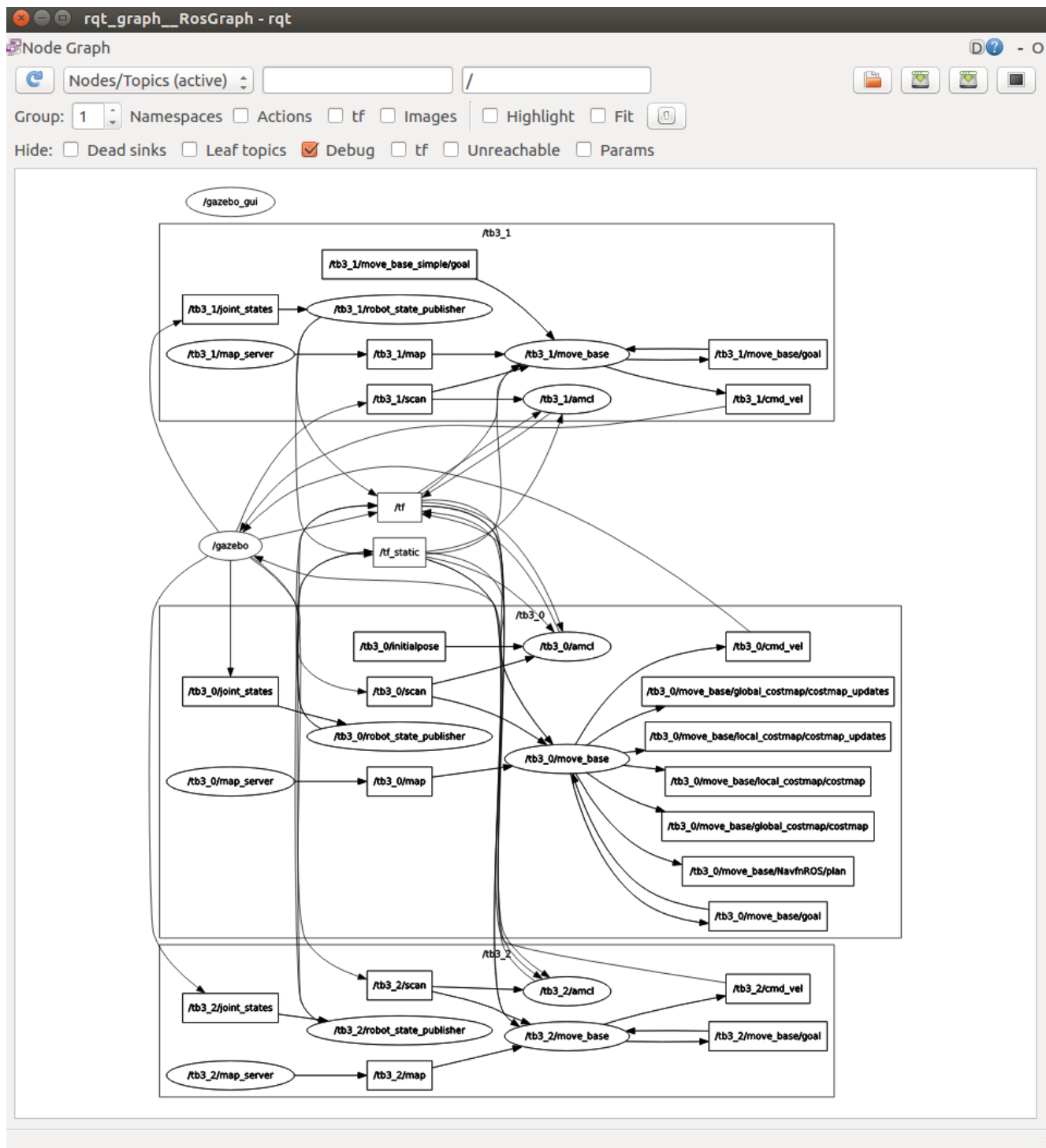


Rqt Graph

Służy do wyświetlania nodów, topiców z podziałem na aktywne/nieaktywne/wszystkie. Nazwy nodów zapisane są w owalnych kształtach (np. /tb3_1/amcl). Nazwy topiców w pojedynczych prostokątach (np. /tb3_1/cmd_vel). Zgrupowane elementy w jednej przestrzeni nazw zgrupowane są większym prostokątem (np. /tb3_1)

```
In [3]: !roslaunch rqt_graph rqt_graph
```

```
QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '/tmp/run  
time-ubuntu'  
^C
```

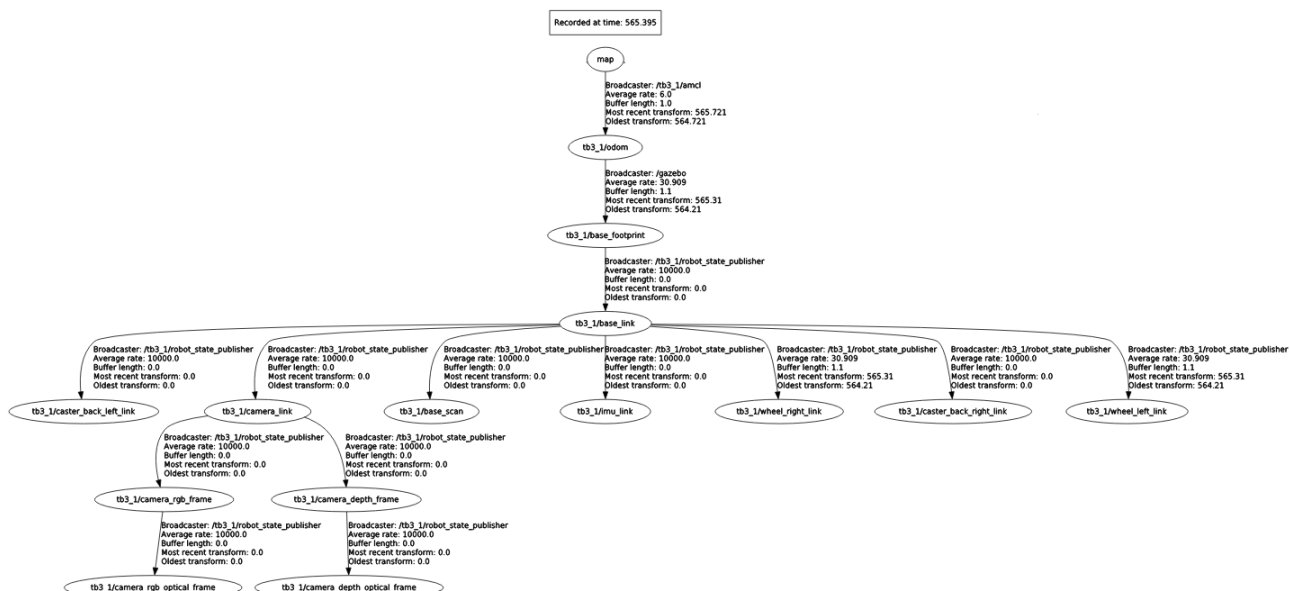


Rqt tf tree

Służy do wyświetlania układów współrzędnych TF i zależności pomiędzy nimi.

In [3]: !roslaunch rqt_tf_tree rqt_tf_tree

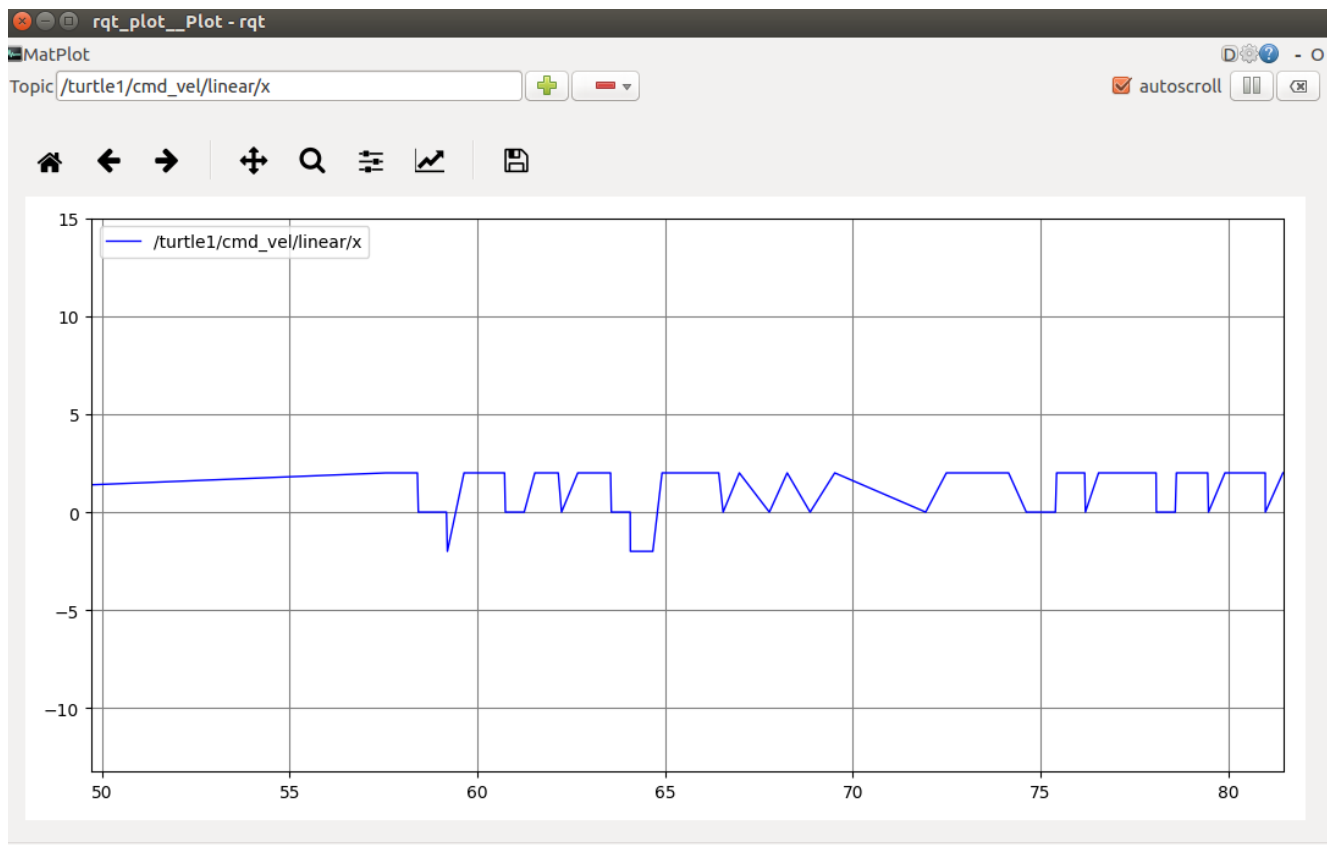
```
QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '/tmp/run
time-ubuntu'
/opt/ros/noetic/lib/python3/dist-packages/rqt_tf_tree/dotcode_t
f.py:96: YAMLLoadWarning: calling yaml.load() without Loader=...
is deprecated, as the default Loader is unsafe. Please read http
s://msg.pyyaml.org/load for full details.
data = yaml_parser.load(yaml_data)
```



Rqt plot

In [5]: !roslaunch rqt_plot rqt_plot

```
QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '/tmp/run
time-ubuntu'
/opt/ros/noetic/lib/python3/dist-packages/rqt_plot/data_plot/mat
_data_plot.py:123: UserWarning: Tight layout not applied. The bo
ttom and top margins cannot be made large enough to accommodate
all axes decorations.
self.figure.tight_layout()
```

In []: